

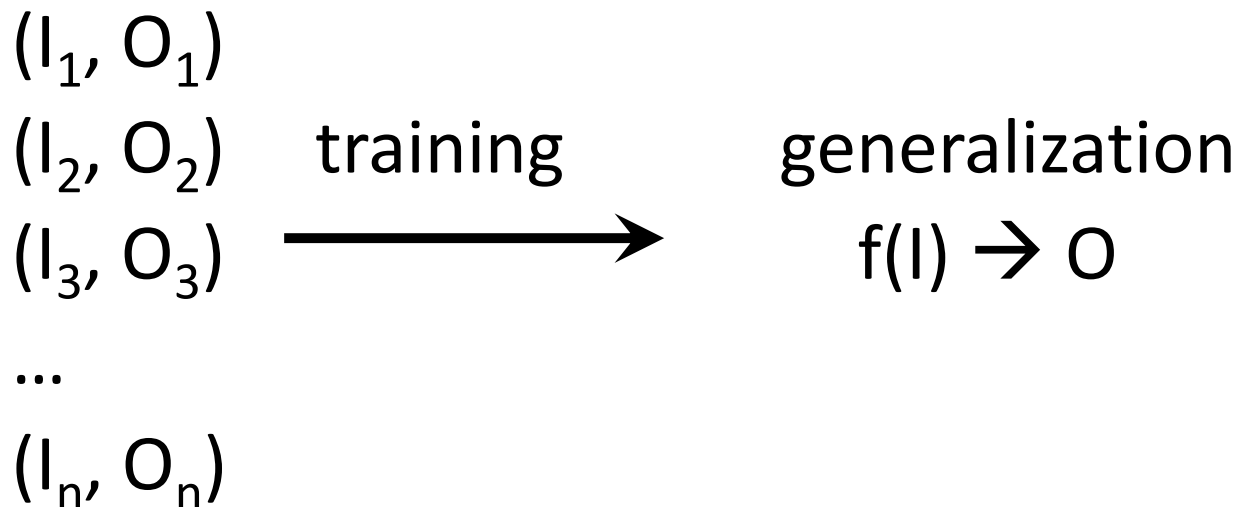
Text categorization (classification)

Classification (categorization)

- many text mining tasks require that the data items to be processed have assigned category labels => importance of classification
- a label
 - symbolic, no additional knowledge is available
 - characterizes (gives a name to) a class (category)
- applications: document (web page, newspaper article, scientific paper, ...) categorization, spam detection, word sense disambiguation, authorship attribution, language identification, sentiment analysis, etc.

Classification problem

- to find a function (classifier, hypothesis) that can be, according to training data, represented by input data (training examples) paired with desired outputs (class labels), used for predicting the outputs for new, unseen data



Would you like to loose weight?	SPAM
---------------------------------	------

Cheap Viagra!!!	SPAM
-----------------	------

Amateur teen girls	SPAM
--------------------	------

Urgent money transfer	SPAM
-----------------------	------

...

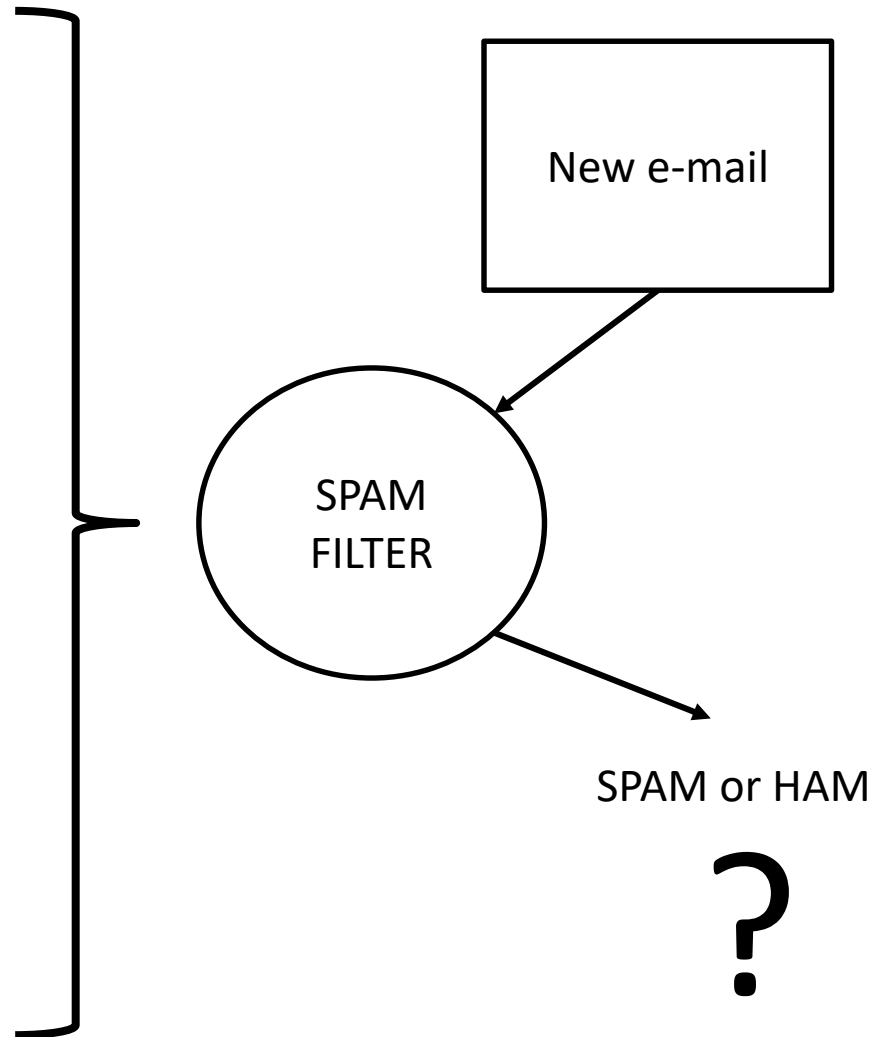
Department meeting takes place on Tuesday	HAM
---	-----

Registration of courses has started	HAM
-------------------------------------	-----

New grant possibilities have been announced	HAM
---	-----

Dissertation thesis defense	HAM
-----------------------------	-----

...



Classification problem

- having documents $d_1, d_2, \dots, d_n$ and classes $c_1, c_2, \dots, c_m$, the hypothesis assigns true or false to each pair (d_j, c_i) denoting whether document d_j belongs to category c_i
- alternatively, the categories might be ranked according to their appropriateness and a human expert might decide on a category (categories)

Classification problem

- single-label classification – only one category is assigned
- binary classification – a document belongs to c_i or to the complement of c_i
- multi-label classification
 - a document can belong to $0..|C|$ categories
 - can be transformed to $|C|$ independent binary classifications

Classifier effectiveness measures

Classifier effectiveness measures

- standard measures, like the accuracy, precision, and recall, might be used
- precision: $P(f(c_i, d_x) = \text{true} \mid h(c_i, d_x) = \text{true})$
 - the probability that if a random document d_x is classified under c_i , the decision is correct
- recall: $P(h(c_i, d_x) = \text{true} \mid f(c_i, d_x) = \text{true})$
 - the probability that if a random document d_x should be classified under c_i , this decision is taken

Classifier effectiveness measures

- the values of the precision and recall can be estimated from the contingency table (P...predicted, R...real)

Class 1	P+	P-
R+	50	30
R-	15	110

$$\text{Prec} = 50 / (50 + 15)$$

$$\text{Rec} = 50 / (50 + 30)$$

Class 2	P+	P-
R+	60	20
R-	20	105

$$\text{Prec} = 60 / (60 + 20)$$

$$\text{Rec} = 60 / (60 + 20)$$

Class 3	P+	P-
R+	40	5
R-	20	140

$$\text{Prec} = 40 / (40 + 20)$$

$$\text{Rec} = 40 / (40 + 5)$$

	P Class 1	P Class 2	P Class 3
R Class 1	50	20	10
R Class 2	10	60	10
R Class 3	5	0	40

$$\text{Prec}(C1) = 50 / (50 + 10 + 5)$$

$$\text{Rec}(C1) = 50 / (50 + 20 + 10)$$

$$\text{Prec}(C2) = 60 / (20 + 60 + 0)$$

$$\text{Rec}(C2) = 60 / (10 + 60 + 10)$$

$$\text{Prec}(C3) = 40 / (10 + 10 + 40)$$

$$\text{Rec}(C3) = 40 / (5 + 0 + 40)$$

Micro- and macro- averaging

Micro- and macro- averaging

- macroaveraging
 - computes a simple average over classes
 - each class has equal weight
- microaveraging
 - aggregates per-document decisions across classes (computes a measure on the pooled contingency table)
 - classes have different weights depending on their size (bigger classes contribute to a measure value more)

Micro- and macro- averaging

Class 1		
	R: yes	R: no
P: yes	10	10
P: no	10	970

Class 2		
	R: yes	R: no
P: yes	90	10
P: no	10	890

Overall		
	R: yes	R: no
P: yes	100	20
P: no	20	1860

R ... real value, P ... predicted value

- $\text{Prec}(1) = 10/(10+10) = 0.5$
- $\text{Prec}(2) = 90/(90+10) = 0.9$
- microaveraged: $\text{Prec}(\text{overall}) = 100/(100+20) = 0.83$
- macroaveraged: $(\text{Prec}(1) + \text{Prec}(2))/2 = 0.7$
- microaveraged Precision is closer to 0.9 than to 0.5 because Class 2 is five times bigger than Class 1

Classifier effectiveness measures

- accuracy and error
 - Accuracy = $(TP + TN) / (TP + FP + TN + FN)$
 - Error = $1 - \text{Accuracy}$
 - problems with unbalanced classes
- F-measure

$$F = \frac{1}{\alpha \frac{1}{P} + (1-\alpha) \frac{1}{R}} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}, \text{ where } \beta^2 = \frac{1-\alpha}{\alpha}$$

Evaluating classifiers

Evaluating classifiers

- the set of documents is split into
 - training (and validation) set T_r – are used to build a classifier
 - test set T_e – is used to evaluate the classifier, i.e., how predicted labels match those assigned by experts
 - training and test sets need to be mutually exclusive (disjoint), otherwise the expectations would be unrealistically good
 - a classifier might be then retrained on the entire set ($T_r + T_e$), the evaluation will be then a pessimistic estimate

Evaluating classifiers

- the mentioned approach is called train-and-test approach
- k-fold cross validation
 - the set of documents D is split into k disjoint sets $D_1, \dots, D_k$
 - the train-and-test approach is applied k times on pairs $D-D_i$ (*training*), D_i (*test*)
 - the results are averaged

Evaluating classifiers

- when some parameters of a classifier need to be tuned, the training set might be further split into training and validation (hold out) sets
- the validation set is used to optimize the parameters
- the test and validation are disjoint so the error of the classifier is not estimated on the data on which it has been optimized

Machine learning classification

- a classifier is built by observing the characteristics of a set of documents manually classified by a domain expert $\rightarrow$ supervised learning problem
- requires the availability of
 - a set of documents $D = \{d_1, \dots, d_{|D|}\}$
 - a set of classes $C = \{c_1, \dots, c_{|C|}\}$
 - a function $F: D \times C \rightarrow \{\text{true}, \text{false}\}$
 - if $F(d_i, c_j) = \text{true}$, d_i is a positive example
 - if $F(d_i, c_j) = \text{false}$, d_i is a negative example

Commonly used classifiers

- probabilistic classifiers, decision trees, decision rules, example-based classifiers, support vectors machine, or neural networks
- it is very difficult to compare individual methods
 - different authors use different sampling, preprocessing, algorithm setting, etc.
 - generally, support vectors machine, instance based classifiers, neural networks, and decision trees bring acceptable results

Probabilistic classifiers

- calculate the probability $P(c_j | d_i)$ that a document d_i belongs to class c_j
- this can be computed using the Bayes' theorem

$$P(c_j | d_i) = \frac{P(c_j)P(d_i | c_j)}{P(d_i)}$$

Naïve Bayes Classifier

- finding the most likely class (maximum a posteriori)

$$C_{MAP} = \operatorname{argmax}_{c_j \in C} \frac{P(c_j)P(d_i|c_j)}{P(d_i)} = \operatorname{argmax}_{c_j \in C} P(c_j)P(d_i|c_j)$$

- $d_i = (x_1, x_2, \dots, x_m)$ so the probability $P(x_1, x_2, \dots, x_m | c_j)$ must be calculated (possible only when a very large number of training examples is available)
- if the features are independent
$$P(x_1, x_2, \dots, x_m | c_j) = P(x_1 | c_j) \cdot P(x_2 | c_j) \cdot \dots \cdot P(x_m | c_j)$$

Naïve Bayes Classifier

- calculating the necessary probabilities – from frequencies in the data

$$P(c_j) = \frac{\textit{number of documents from class } c_j}{\textit{number of documents}}$$

$$P(x_i | c_j) = \frac{\textit{count}(x_i, c_j)}{\sum_{x \in V} \textit{count}(x, c_j)}$$

Naïve Bayes Classifier

- if one of $P(x_i|c_j)$ is 0 the entire formula will result in zero, no matter the other evidence
- thus, Laplace smoothing is often used

$$\begin{aligned} P(x_i|c_j) &= \frac{\text{count}(x_i, c_j) + 1}{\sum_{x \in V} (\text{count}(x, c_j) + 1)} \\ &= \frac{\text{count}(x_i, c_j) + 1}{(\sum_{x \in V} \text{count}(x, c_j)) + |V|} \end{aligned}$$

$|V|$... size of the vocabulary

Naïve Bayes Classifier – example

good	+
very good	+
very very good	+
not good	-
very bad	-
bad	-
is not very good	?

is is not in the training examples, is ignored

we need to calculate $P(? | \text{not very good})$, where ? is + and -

we need $P(+)$, $P(-)$, $P(\text{not very good} | +)$, and $P(\text{not very good} | -)$

Naïve Bayes Classifier – example

good	+
very good	+
very very good	+
not good	-
very bad	-
bad	-
not very good	?

$P(+) = 3/6$
$P(-) = 3/6$
$P(\text{good} +) = (3+1)/(6+4)$
$P(\text{very} +) = (3+1)/(6+4)$
$P(\text{bad} +) = (0+1)/(6+4)$
$P(\text{not} +) = (0+1)/(6+4)$
$P(\text{good} -) = (1+1)/(5+4)$
$P(\text{very} -) = (1+1)/(5+4)$
$P(\text{bad} -) = (2+1)/(5+4)$
$P(\text{not} -) = (1+1)/(5+4)$

$V = \{\text{good, very, not, bad}\}$

Naïve Bayes Classifier – example

good	+
very good	+
very very good	+
not good	-
very bad	-
bad	-
not very good	?

$P(+) = 3/6$
$P(-) = 3/6$
$P(\text{good} +) = (3+1)/(6+4)$
$P(\text{very} +) = (3+1)/(6+4)$
$P(\text{bad} +) = (0+1)/(6+4)$
$P(\text{not} +) = (0+1)/(6+4)$
$P(\text{good} -) = (1+1)/(5+4)$
$P(\text{very} -) = (1+1)/(5+4)$
$P(\text{bad} -) = (2+1)/(5+4)$
$P(\text{not} -) = (1+1)/(5+4)$

$$P(+)*P(\text{not very good} | +) = 1/2 * 1/10 * 4/10 * 4/10 = 0.008$$

$$P(-)*P(\text{not very good} | -) = 1/2 * 2/9 * 2/9 * 2/9 = 0.005487$$

Naïve Bayes Classifier – example

good	+
very good	+
very very good	+
not good	-
very bad	-
bad	-
not very good	?

$P(+) = 3/6$
$P(-) = 3/6$
$P(\text{good} +) = (3+1)/(6+4)$
$P(\text{very} +) = (3+1)/(6+4)$
$P(\text{bad} +) = (0+1)/(6+4)$
$P(\text{not} +) = (0+1)/(6+4)$
$P(\text{good} -) = (1+1)/(5+4)$
$P(\text{very} -) = (1+1)/(5+4)$
$P(\text{bad} -) = (2+1)/(5+4)$
$P(\text{not} -) = (1+1)/(5+4)$

$$P(+)*P(\text{not very good} | +) = 1/2 * 1/10 * 4/10 * 4/10 = 0.008$$

$$P(-)*P(\text{not very good} | -) = 1/2 * 2/9 * 2/9 * 2/9 = 0.005487$$

Naïve Bayes Classifier

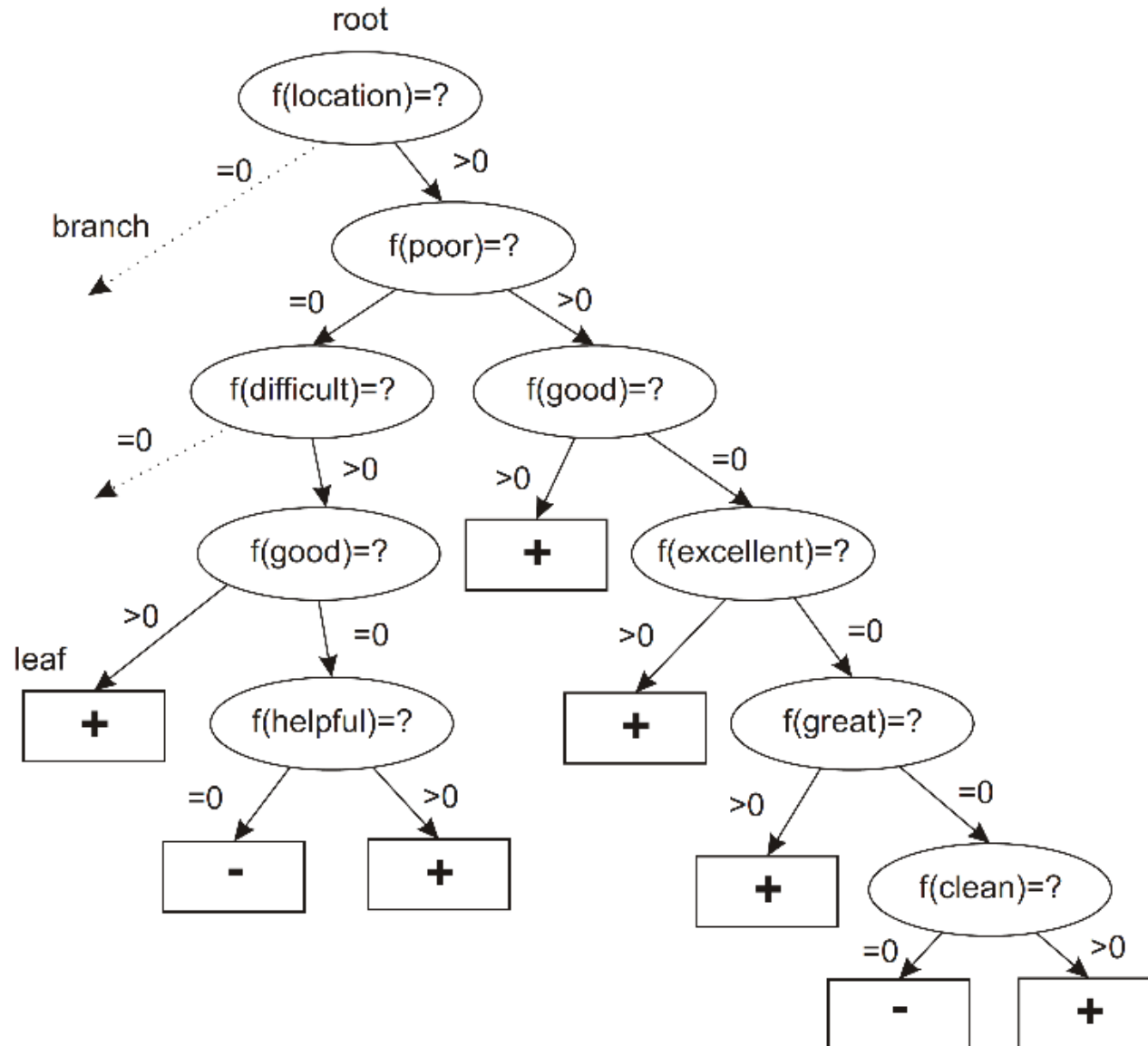
- this was the multinomial NB variant – typical for texts
- binary NB first removes duplicate words from each training (and later also test) document
- very fast method, working well for smaller datasets

Decision trees

Decision trees

- a DT is a tree where
 - the internal nodes are labeled by terms
 - the branches are labeled by tests on the weights of the terms
 - the leaves are class labels
- classification has the form of a sequence of tests taking place until a leaf is reached
- able to explain *why* a certain instance should be assigned to a specific class
- work well with imbalanced data, outliers, suitable for smaller datasets, texts combined with metadata
- trees are often combined in an ensemble (e.g, Random Forest, XGBoost)

Decision trees



Decision tree training

- a DT is usually build using the top-down approach:

Decision tree training

- a DT is usually build using the top-down approach:
 1. find an attribute a_i that best divides T (training set)
 2. divide T into subsets T_i , each characterized by a different value of a_i
 3. if all documents in T_i belong to the same class, create a leaf labelled with that class; otherwise, recursively apply the same procedure starting with step 1 to T_i

Decision tree training

- calculating the suitability of an attribute to split the training set – a common measure is the information gain (or gain ratio)

$$IG(T, a_i) = H(T) - H(T, a_i)$$

$$H(T) = - \sum_{j=1}^n p(c_j) \log p(c_j)$$

$$H(T, a_i) = \sum_i P_i H(T_i)$$

H ... entropy, P_i ... relative size of T_i , n ... number of classes

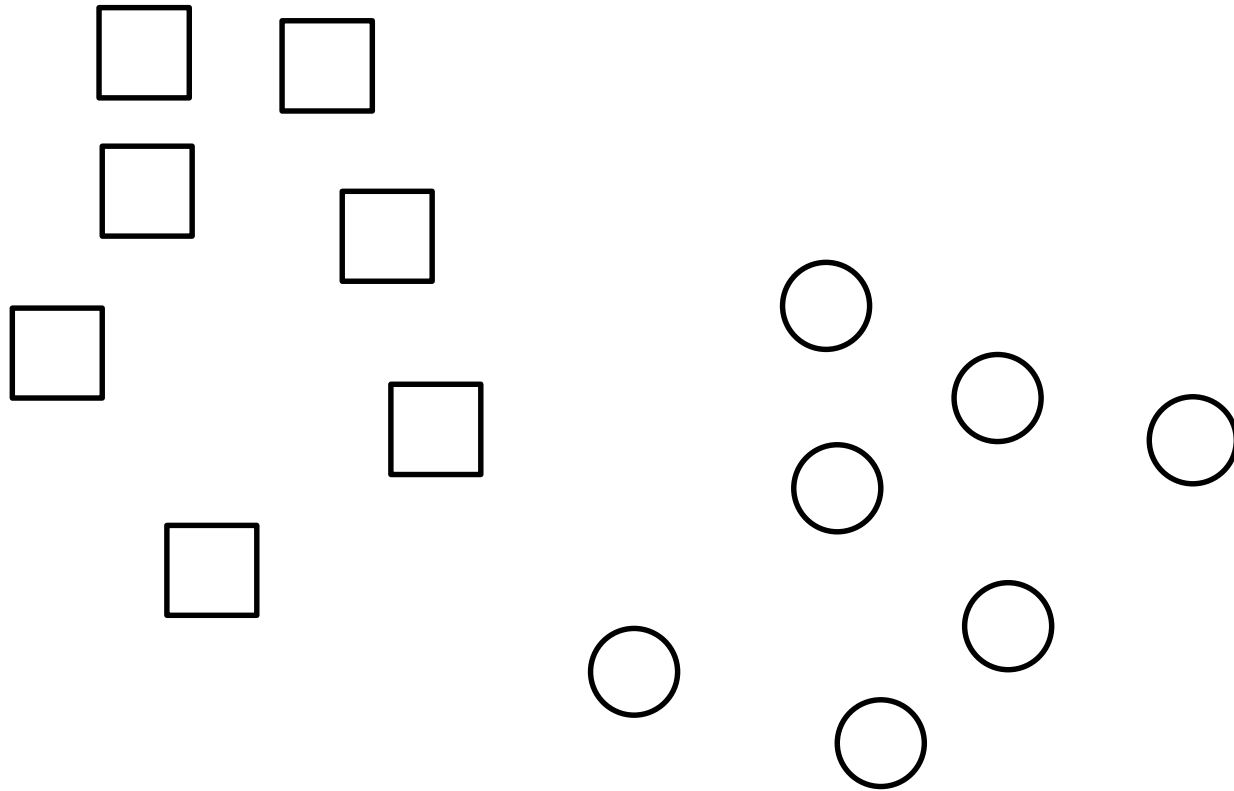
Support vector machine

Support vector machine

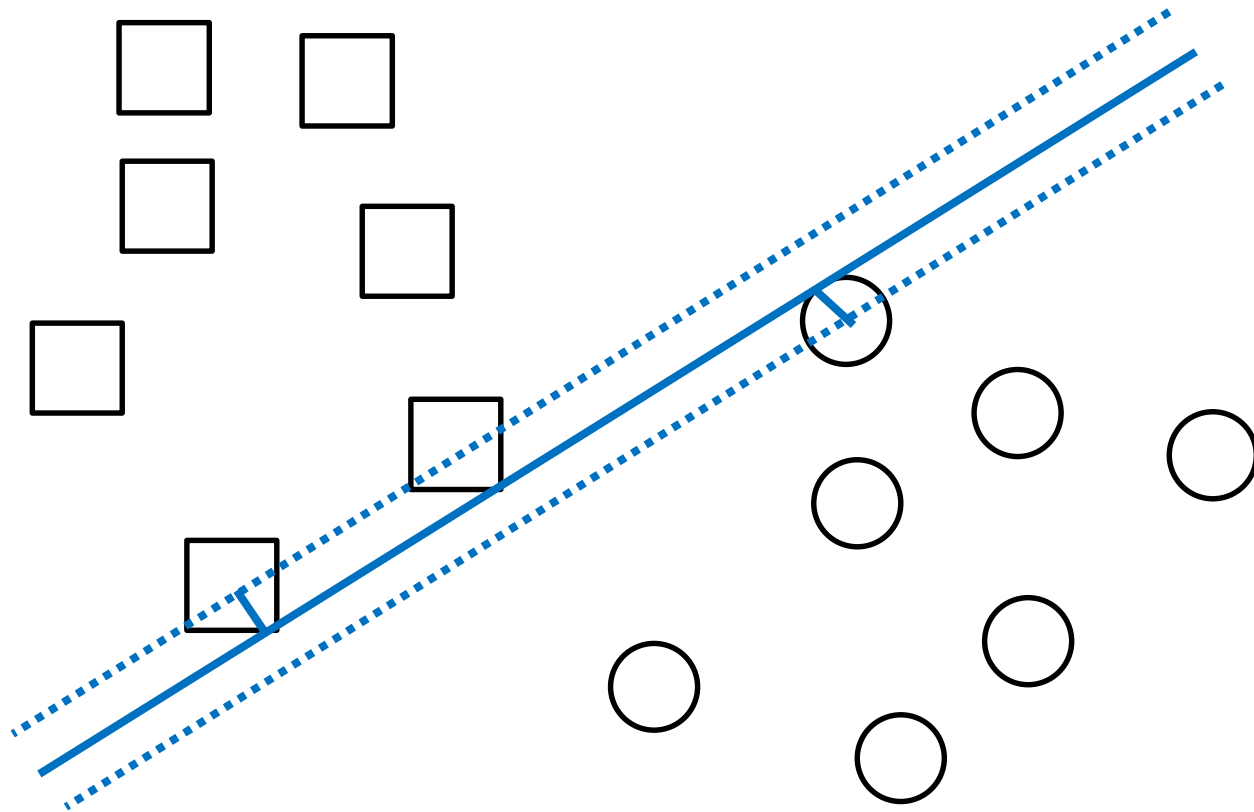
- the data is partitioned by finding a linear boundary (hyperplane) between two classes
- the margin widths between the class boundary and training patterns are maximized during the training process
- the best decision surface is determined by only a small set of training examples, called the support vectors
- robust to overfitting

Support vector machine

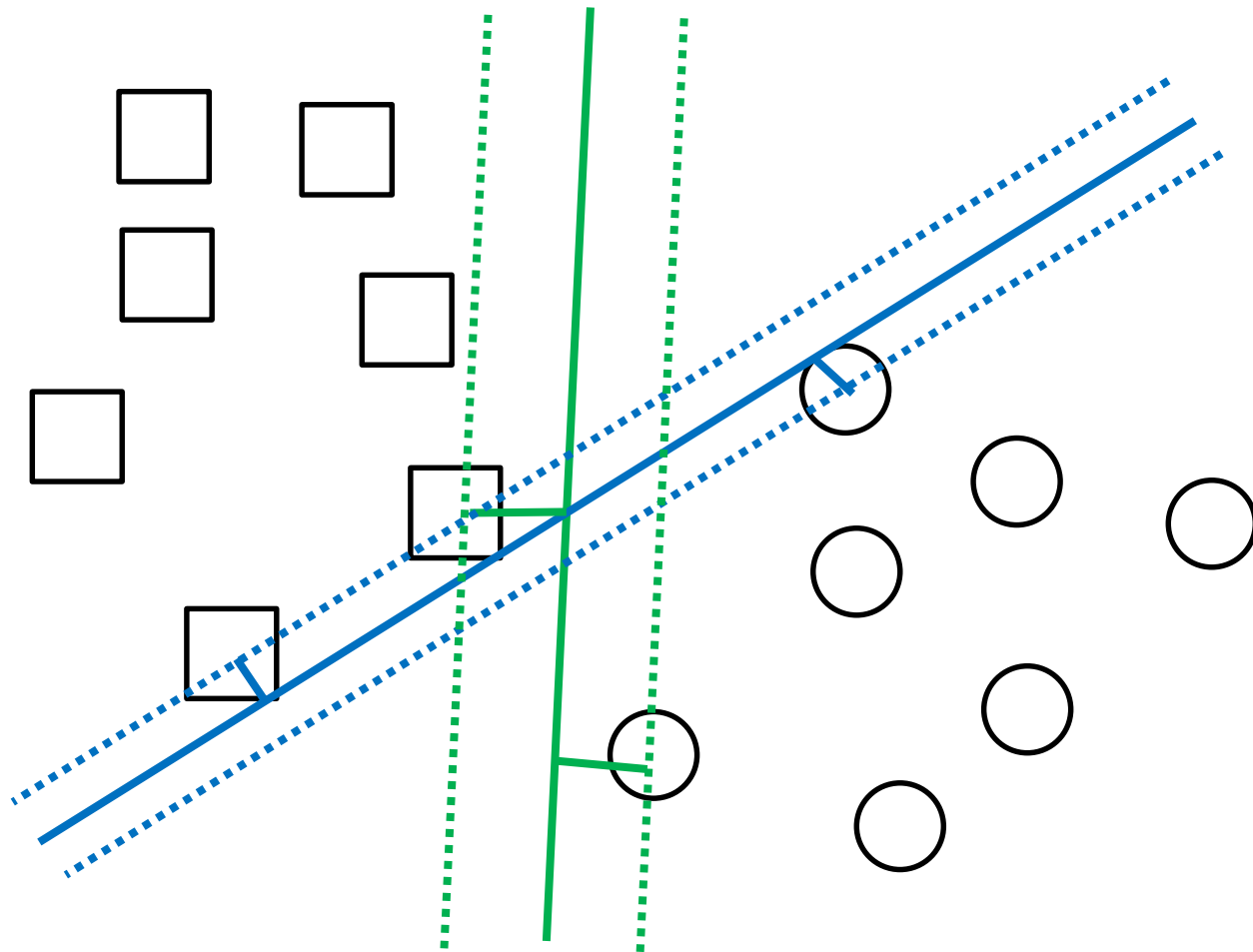
- suitable for high-dimensional, sparse data, and linearly separable (even in higher dimensions) data
- achieved state-of-the-art results before transformer-based models



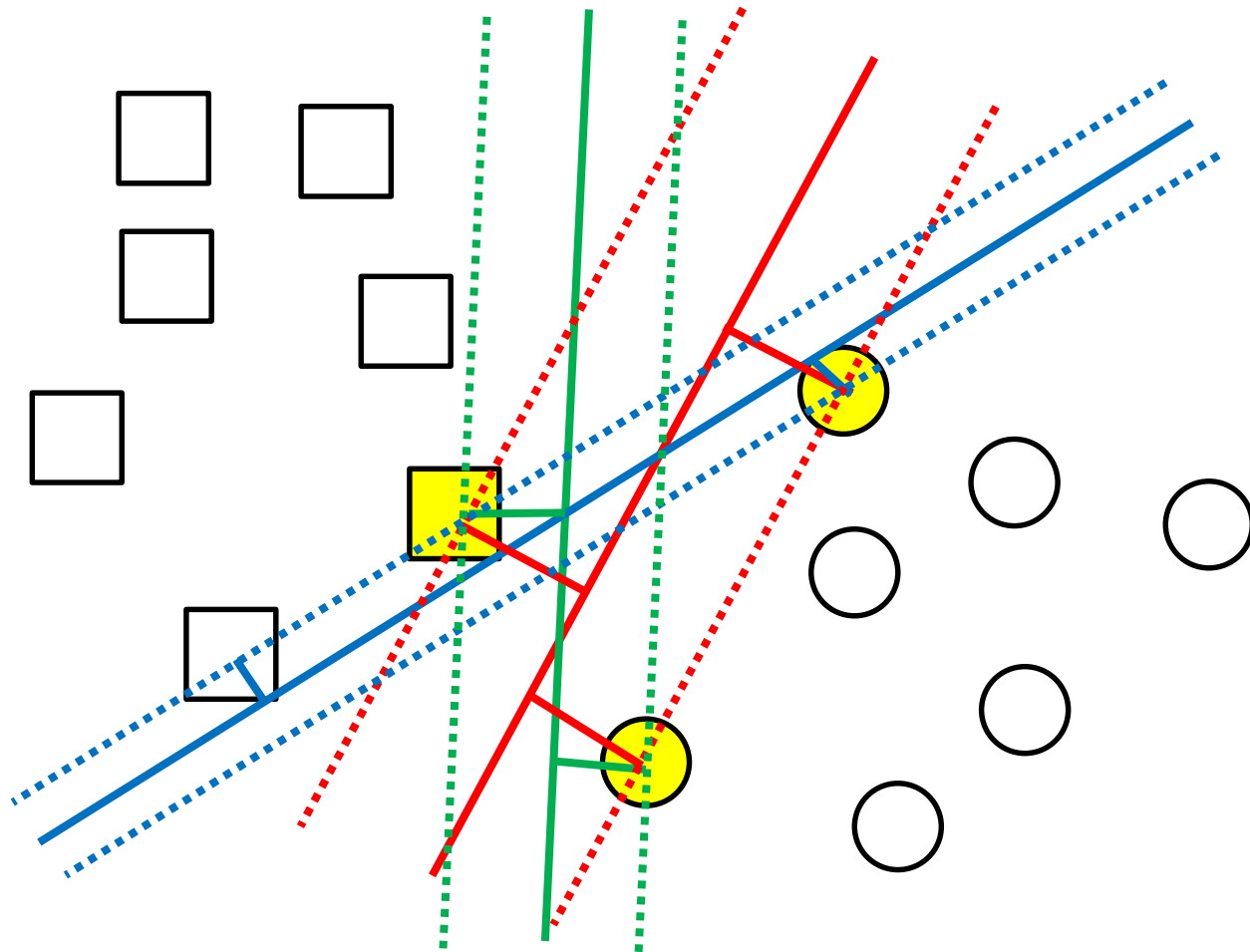
objects of two classes to be separated



the blue line separates the data with some margin



the green line separates the data with a larger margin

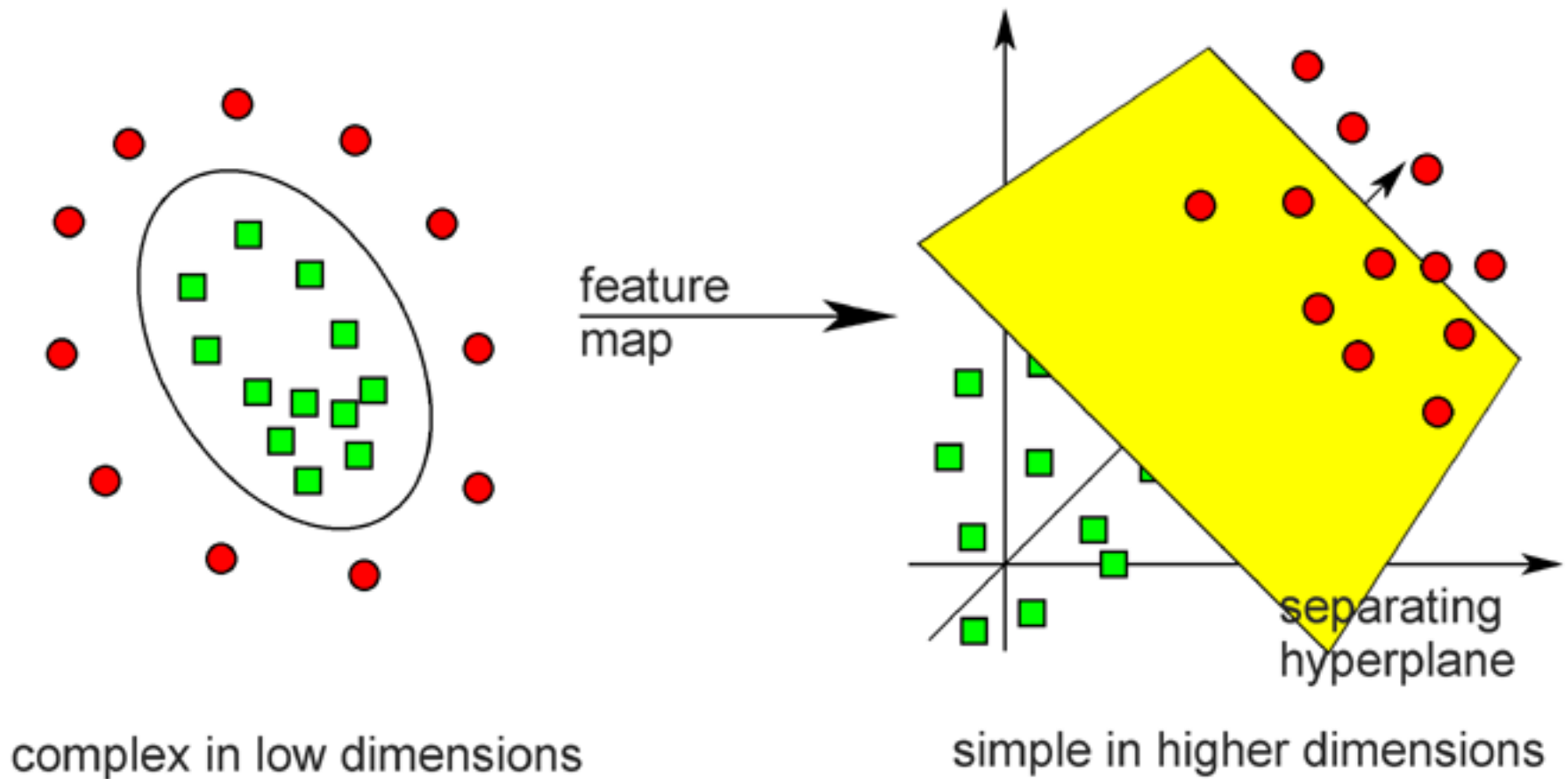


the red line separates the data with the largest margin
(dotted lines contain the support vectors – yellow objects)

Support vector machine

- when it is not possible to separate the data in a given n -dimensional space linearly, a kernel function that projects the data to a space of higher dimension is used

Separation may be easier in higher dimensions



Logistic regression

Logistic regression

- a statistical method for describing a relationship between several variables and one binary one
- very popular in social and natural sciences
- is a basis for neural networks (a NN is a sequence of LR classifiers stacked on top each other)
- calculates the probability that an object should be classified under a class

Logistic regression

- suitable when an explanation of decisions is needed (provides feature importance) and a probability is needed

Making a decision

- every feature x_i has associated a weight + there is a bias term, all are real numbers
- the weights represent importance for classification
 - positive number – the feature is important for assigning to the positive class
 - negative number – the feature is important for assigning to the negative class

Making a decision

- when classifying
 - feature values are multiplied by the weights and the bias term is added

$$z = \sum_{i=1}^m w_i x_i + b = w \cdot x + b$$

- the value is transformed to probability using the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^z}$$

- a threshold for $\sigma(z)$, e.g., 0.5, is used when classification is to be made

$$class = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

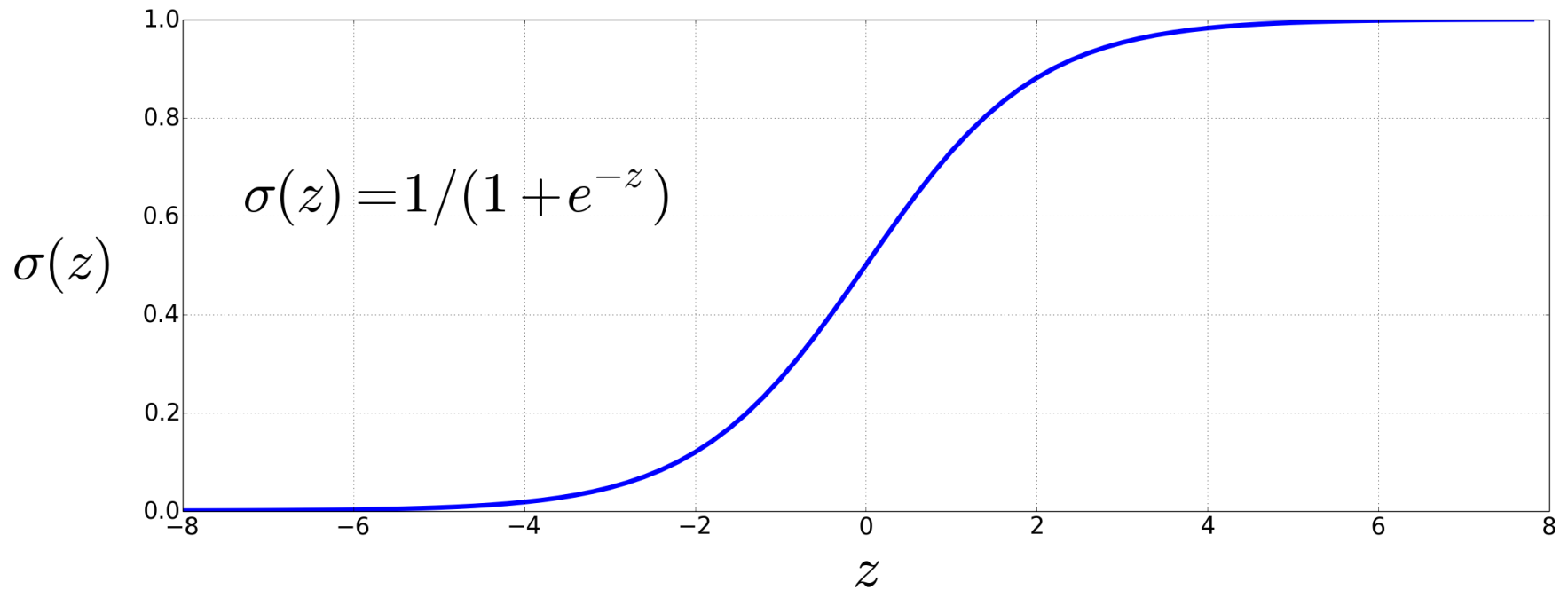
Making a decision

- the Naïve Bayes classifier strongly assumes independence between features
- LR can handle it – if x_i and x_j are correlated, part of the weight will be added to x_i and part to x_j

The sigmoid

- all outputs are in the interval $[0, 1]$
 - $P(y = 1|x) = \sigma(z)$
 - $P(y = 0|x) = 1 - \sigma(z) = \sigma(-(z))$
 - $P(y|x) = \sigma(z)^y \cdot (1 - \sigma(z))^{1-y}$
- the values are
 - almost linear in the middle
 - almost flat at the ends (even for outliers)
- the function is differentiable – suitable for learning

The sigmoid



Source: Jurafsky, Martin: Speech and Language Processing (3rd ed. draft)

Learning

- the goal is to optimize the loss function measuring the correctness of classification on training data
- the loss function is called cross-entropy loss (goes from 0=no loss to infinity):

$$L_{CE}(w, b) = -\sum_{i=1}^n y_i \cdot \log(\sigma(wx_i + b)) + (1 - y_i) \cdot \log(1 - \sigma(wx_i + b))$$

- there is no closed-form solution => gradient descent
- the weights + bias are gradually updated to optimize the loss function

Traditional models

- fast, low cost, sometimes interpretable
- may
 - serve as a baseline
 - make easy decisions (e.g., when a LR model predicts 99%), the other pass to a complex model
 - process embeddings from LLMs + additional tabular data (traditional models are good in processing tabular data)

Artificial neural networks learning

- ANN inputs – features
- ANN outputs – classes
- learning process – updating the inter-neuronal synaptic weights during training iterations in order to maximize the correctness of assigned classes

ANN in NLP

- great progress in last few years due to
 - increased computing power of computers
 - new representations of features having lower dimensionality (embeddings)
 - new approaches to computations that enable, e.g., parallelization

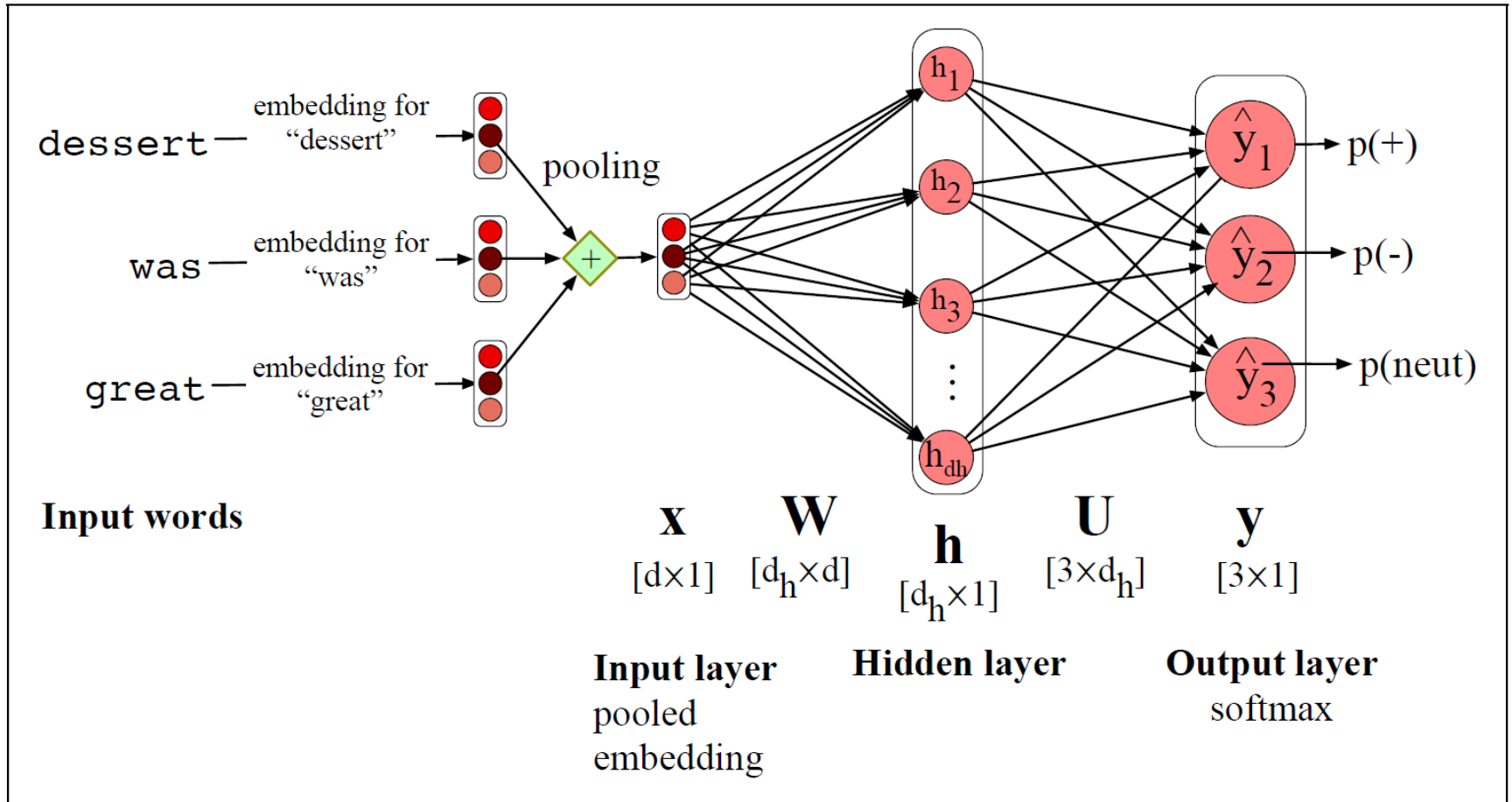


Figure 7.11 Feedforward network sentiment analysis using a pooled embedding of the in-

ANN in NLP

- more complex ANN architectures are popular
 - (deep) MLP – many layers can learn suitable representations
 - CNN (convolutional): adapted from computer vision, can capture local patterns and reduce the number of ANN parameters

ANN in NLP

- more complex ANN architectures are popular
 - RNN (recurrent): can work with sequences of inputs; can be also bidirectional; include architectures like LSTM or GRU to work with long dependencies
 - sequence-to-sequence models: consist of an encoder that learns input representations and a decoder that produces output

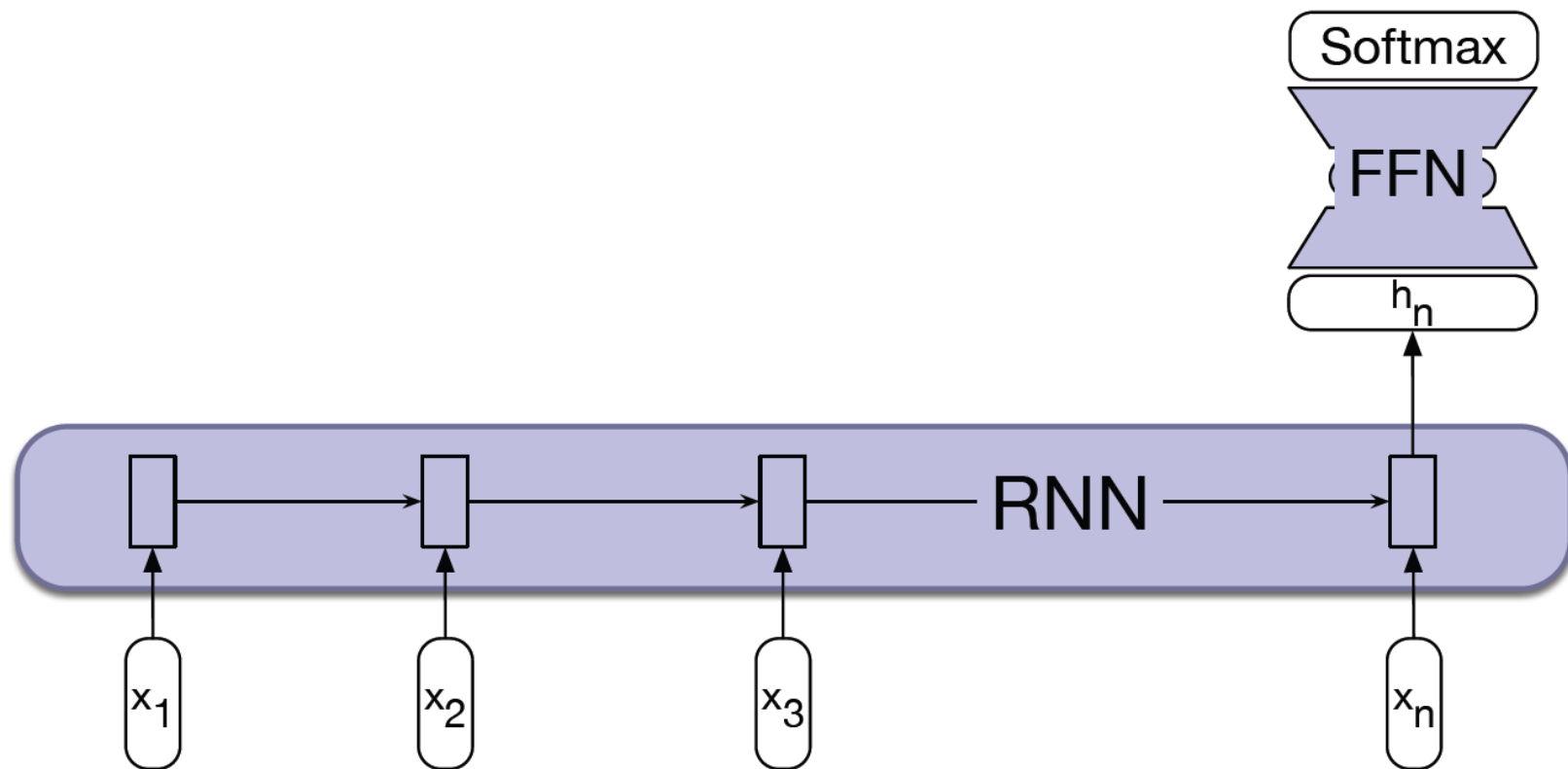


Figure 8.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

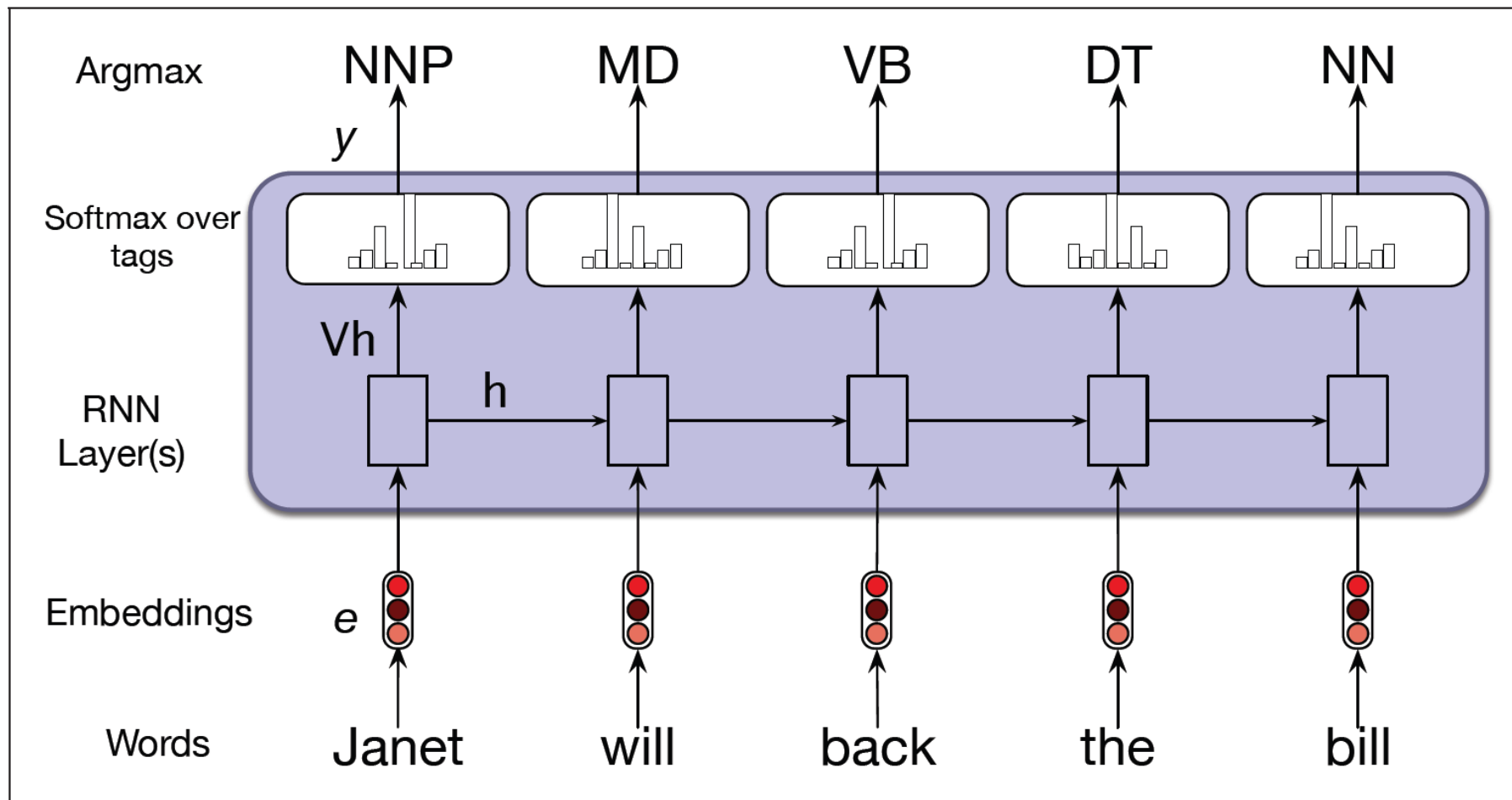


Figure 8.7 Part-of-speech tagging as sequence labeling with a simple RNN.